EWB CLOUD LABS

AWS VPC Peering

Connect two VPCs and prove private connectivity

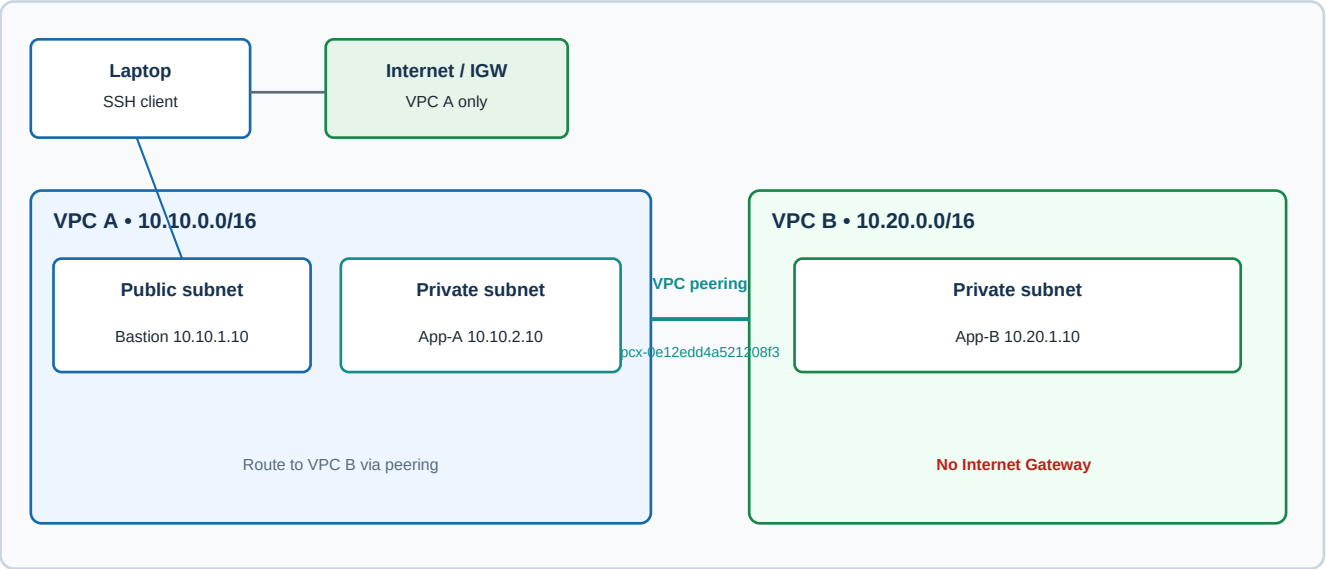
Student	Jaideep
Batch	AWS Internship Batch 1
Region	Asia Pacific (Sydney) ap-southeast-2
Date	29 Oct 2026
AWS Account ID	715844024215
Tags used	Project=EWB-VPC-Peering; Student=Jaideep

All screenshots taken in Asia Pacific (Sydney) ap-southeast-2

Report based on the lab configuration and test evidence shown in this document.

2. Architecture and resource inventory

Two private network ranges are connected using VPC peering. A bastion host in VPC A is the SSH entry point. App-A and App-B use private IPv4 addresses.



Resource inventory

Resource	Name	CIDR / address	ID / notes
VPC	ewb-vpc-a	10.10.0.0/16	vpc-07336a6e7709e442d
VPC	ewb-vpc-b	10.20.0.0/16	vpc-0e459e7953bf98b60
Subnet	ewb-a-public	10.10.1.0/24	subnet-0b12e7bc9c4bbd2ae
Subnet	ewb-a-private	10.10.2.0/24	subnet-0beb6b6f6361f3c15
Subnet	ewb-b-private	10.20.1.0/24	subnet-0b712faffb37caf6a
Internet gateway	ewb-igw-a	VPC A only	igw-02459cd6ad9578815
Route table	ewb-rt-a-public	Public subnet	rtb-0ab2db6b318c3145e
Route table	ewb-rt-a-private	Private subnet	rtb-0e283b77a0c7d0e8e
Route table	ewb-rt-b-private	Private subnet	rtb-0b55bf247751e890a
Security group	ewb-sg-bastion	SSH	sg-09ac2a3aa9d28dcd8
Security group	ewb-sg-app-a	SSH + ICMP	sg-078e381b8fc9374c9
Security group	ewb-sg-app-b	SSH + ICMP	sg-013c958be7b55067c
Peering	ewb-a-b-peer	Active	pcx-0e12edd4a521208f3
Key pair	ewb-lab	EC2 access	Private key not included
EC2	ewb-bastion	10.10.1.10; public 15.135.166.203	i-016fce1eeff400cce; Amazon Linux 2023; t3.micro
EC2	ewb-app-a	10.10.2.10; no public IP	i-04e8a6f9dbb211d8f; Amazon Linux 2023; t3.micro
EC2	ewb-app-b	10.20.1.10; no public IP	i-0930afdf2466795cc; Amazon Linux 2023; t3.micro

3. Subnet CIDRs and route tables

Each subnet is a /24 within its VPC's /16 range. Both private route tables need a route to the other VPC through peering. The public route table also has a default route to VPC A's internet gateway.

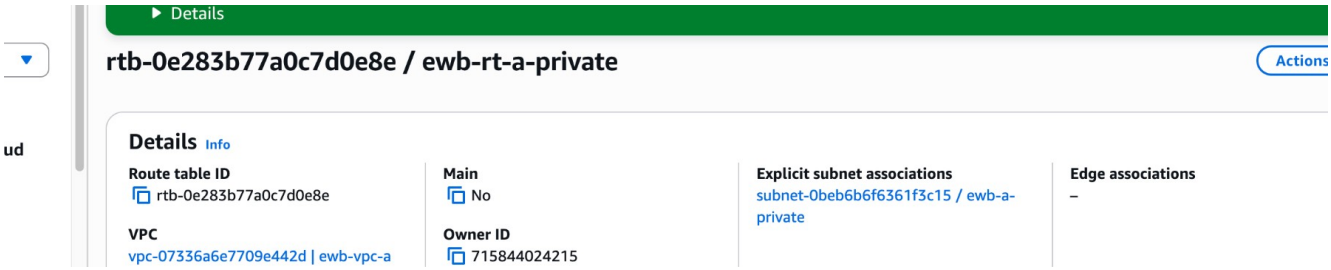
Subnet associations

Subnet	CIDR	Route table	Association
ewb-a-public	10.10.1.0/24	ewb-rt-a-public	Explicit
ewb-a-private	10.10.2.0/24	ewb-rt-a-private	Explicit
ewb-b-private	10.20.1.0/24	ewb-rt-b-private	Explicit

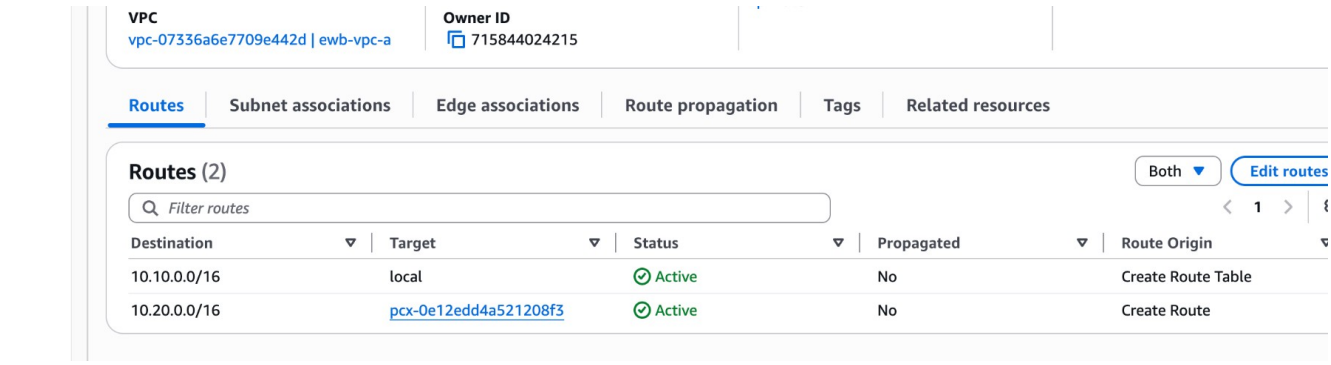
Routes (verified in screenshots)

Route table	Destination	Target
ewb-rt-a-private	10.10.0.0/16	local
ewb-rt-a-private	10.20.0.0/16	pcx-0e12edd4a521208f3
ewb-rt-b-private	10.20.0.0/16	local
ewb-rt-b-private	10.10.0.0/16	pcx-0e12edd4a521208f3
ewb-rt-a-public	10.10.0.0/16	local
ewb-rt-a-public	0.0.0.0/0	igw-02459cd6ad9578815
ewb-rt-a-public	10.20.0.0/16	pcx-0e12edd4a521208f3

3a. Private route restoration evidence

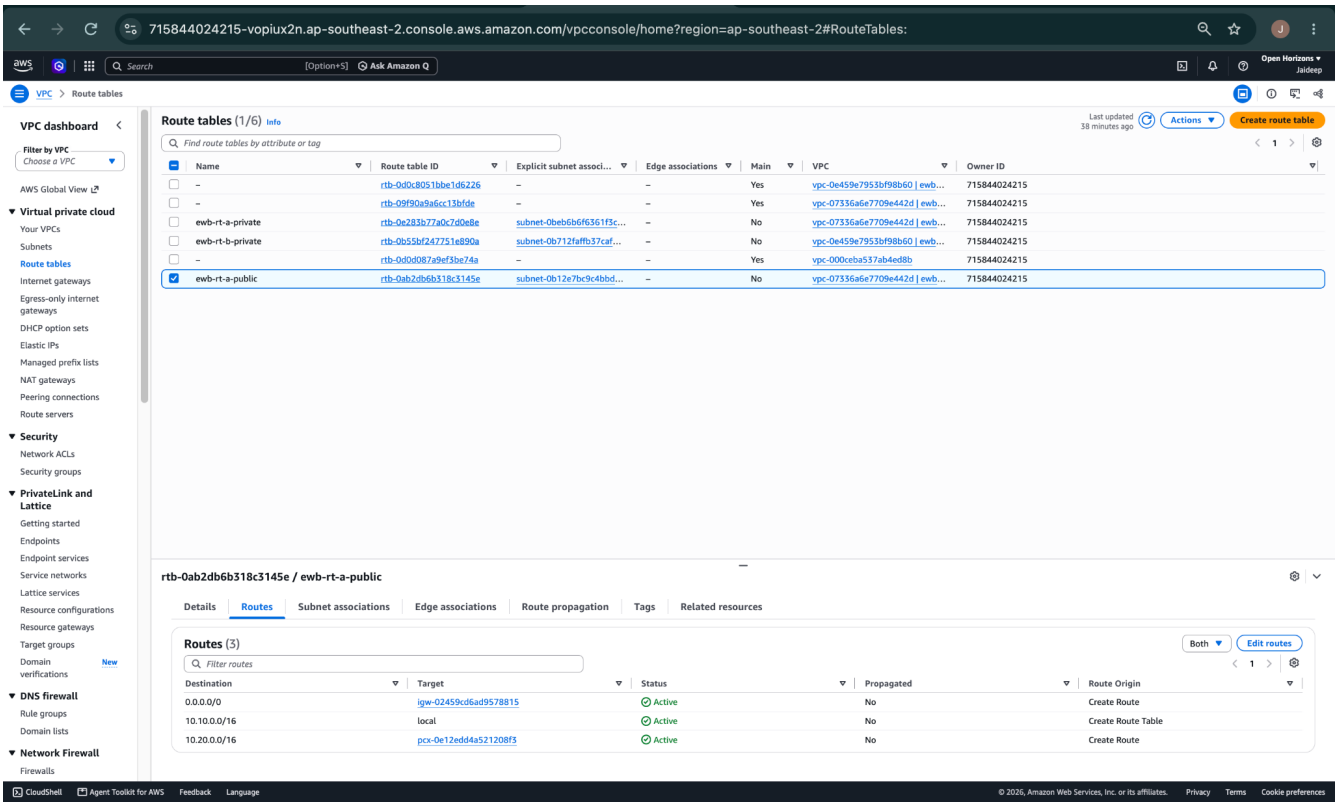


Host: AWS Console | **Test:** Restore ewb-rt-a-private peering route — AWS console table / upper configuration | **Result:** Green update banner shown; private route table displayed



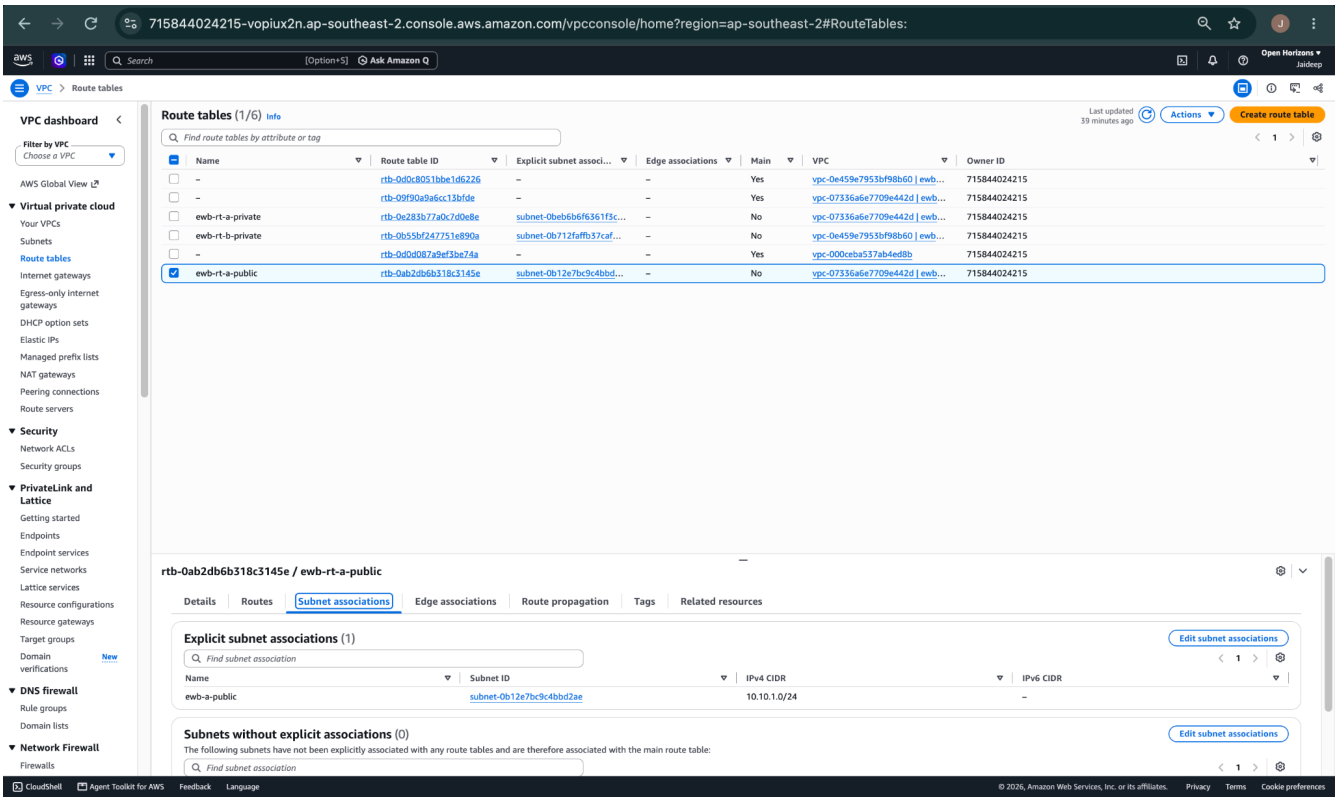
Host: AWS Console | **Test:** Restore ewb-rt-a-private peering route — Selected resource details / lower panel | **Result:** Green update banner shown; private route table displayed

3b. ewb-rt-a-public Routes



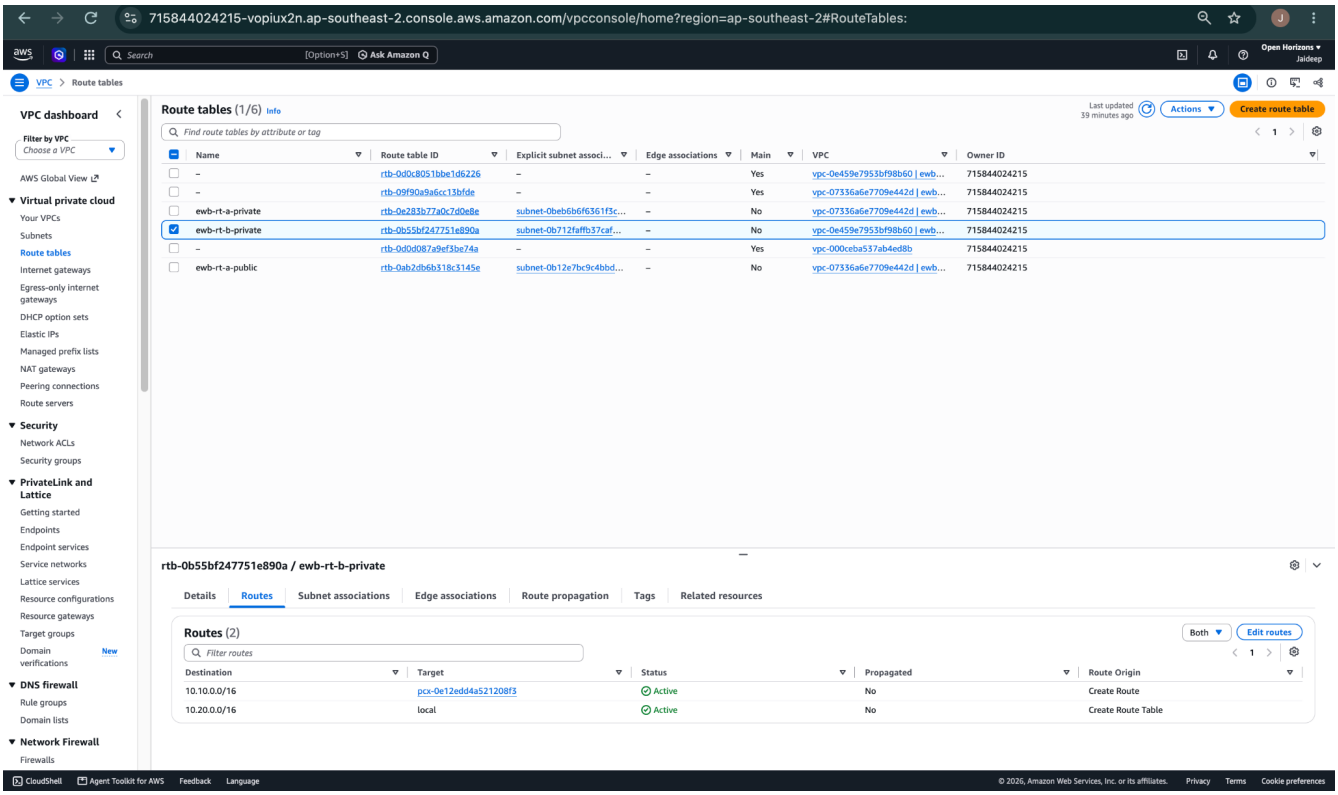
Host: AWS Console | Test: ewb-rt-a-public Routes
Result: Screenshot shows the public route table and its three routes.

3c. ewb-rt-a-public Subnet associations



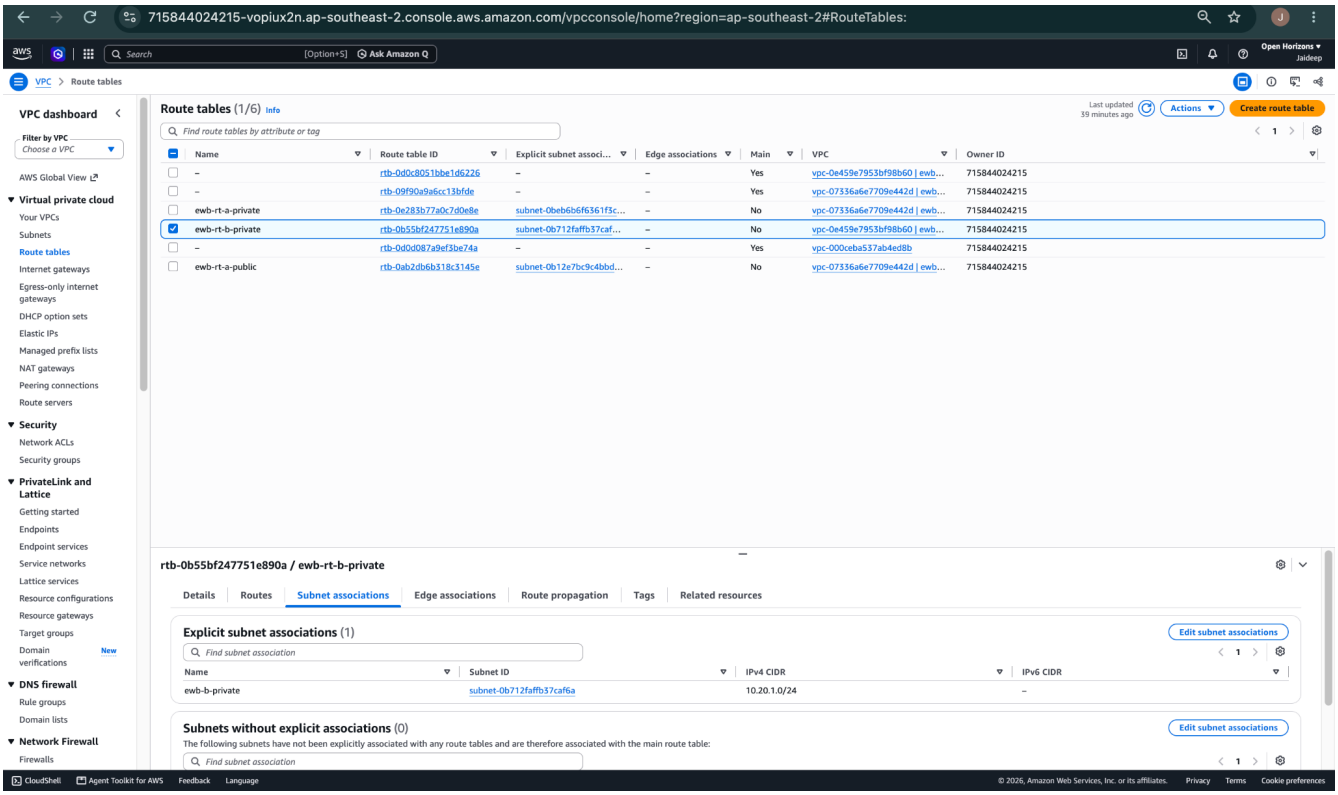
Host: AWS Console | Test: ewb-rt-a-public Subnet associations
Result: Screenshot shows the subnet association.

3d. ewb-rt-b-private Routes



Host: AWS Console | Test: ewb-rt-b-private Routes
Result: Screenshot shows the two routes.

3e. ewb-rt-b-private Subnet associations



Host: AWS Console | Test: ewb-rt-b-private Subnet associations
Result: Screenshot shows the subnet association.

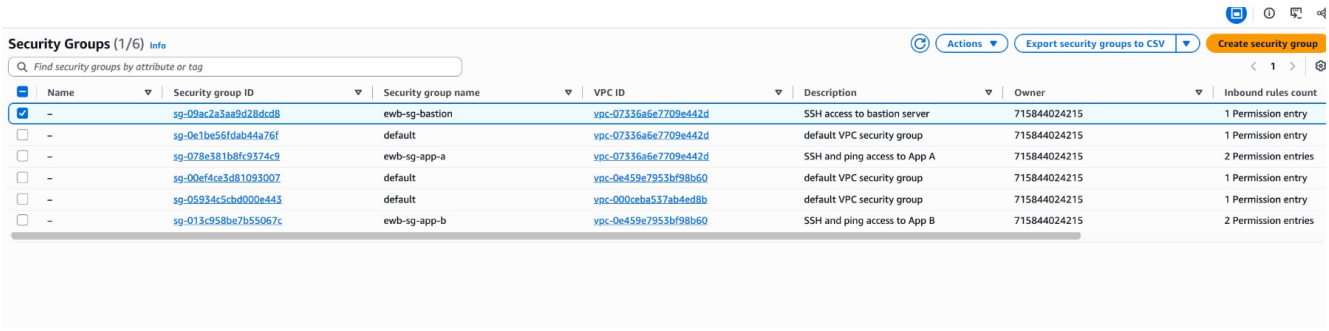
4. Active peering and security groups

Peering is Active, with VPC A as requester and VPC B as accepter. Routes and security group rules must also allow traffic; Active peering alone does not guarantee connectivity.

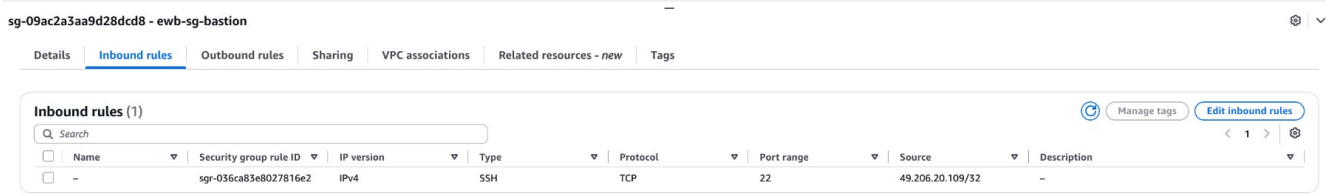
Inbound rules

Security group	Type / port	Source	Rule ID
ewb-sg-bastion	SSH / TCP 22	49.206.20.109/32	sgr-036ca83e8027816e2
ewb-sg-app-a	SSH / TCP 22	10.10.1.10/32	sgr-0dcfdab3bc9c5f07e
ewb-sg-app-a	ICMP Echo Request	10.20.1.10/32	sgr-0dee1e31b5b4996d1
ewb-sg-app-b	SSH / TCP 22	10.10.1.10/32	sgr-07ca07b9072b4b136
ewb-sg-app-b	ICMP Echo Request	10.10.2.10/32	sgr-0051e35a4a30d6de2 (restored; old ID removed)

4a. Bastion security group

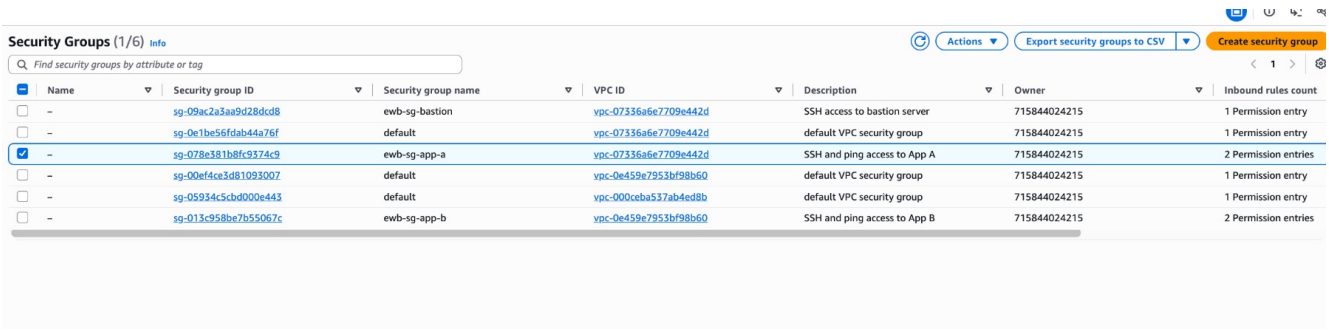


Host: AWS Console | **Test:** ewb-sg-bastion inbound rules — AWS console table / upper configuration | **Result:** 1 SSH rule from 49.206.20.109/32 displayed

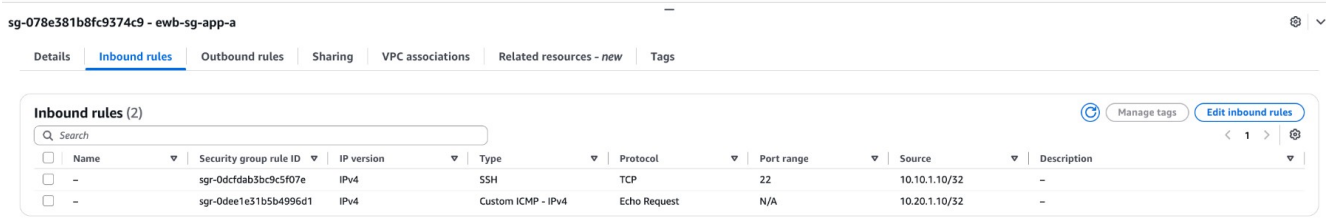


Host: AWS Console | **Test:** ewb-sg-bastion inbound rules — Selected resource details / lower panel | **Result:** 1 SSH rule from 49.206.20.109/32 displayed

4b. App-A security group

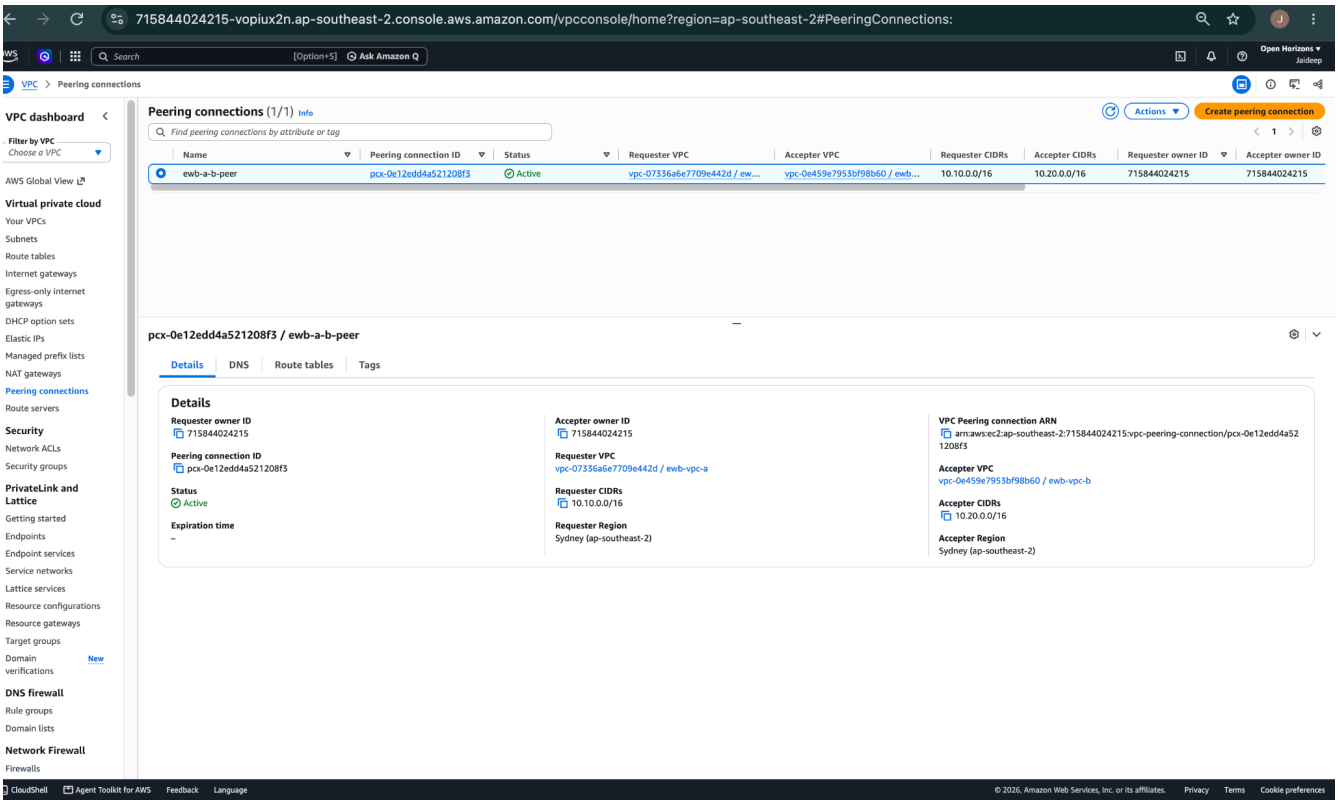


Host: AWS Console | **Test:** ewb-sg-app-a inbound rules — AWS console table / upper configuration | **Result:** 2 inbound rules displayed



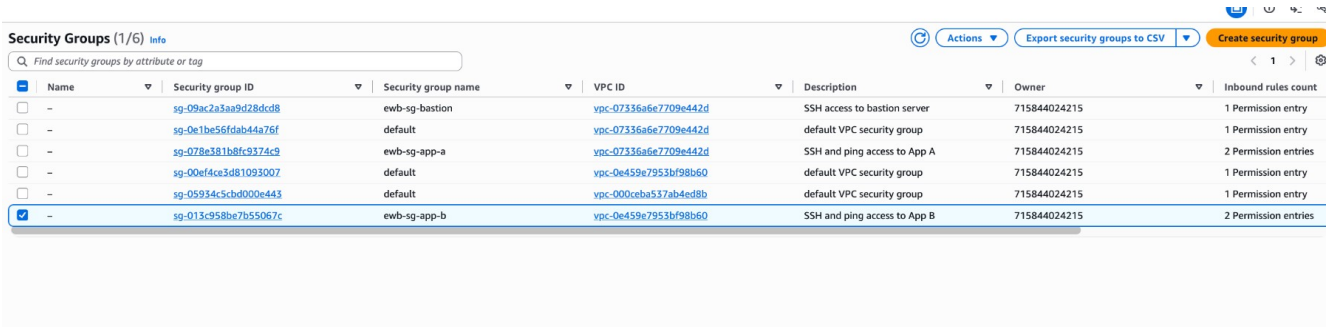
Host: AWS Console | **Test:** ewb-sg-app-a inbound rules — Selected resource details / lower panel | **Result:** 2 inbound rules displayed

4c. Peering connection status

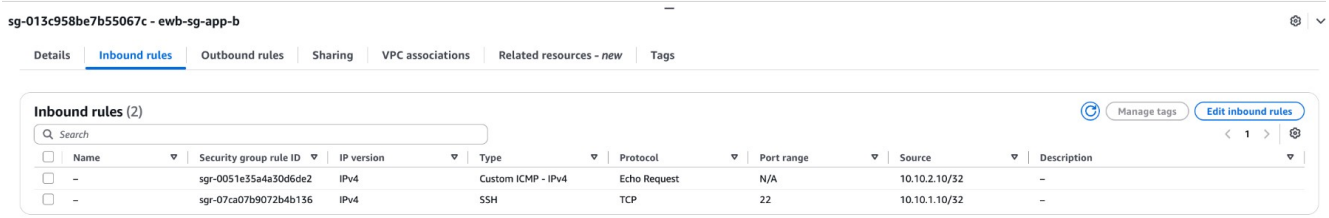


Host: AWS Console | Test: ewb-a-b-peer peering status
Result: Screenshot shows the Active peering connection and VPC IDs.

4d. App-B security group after recovery

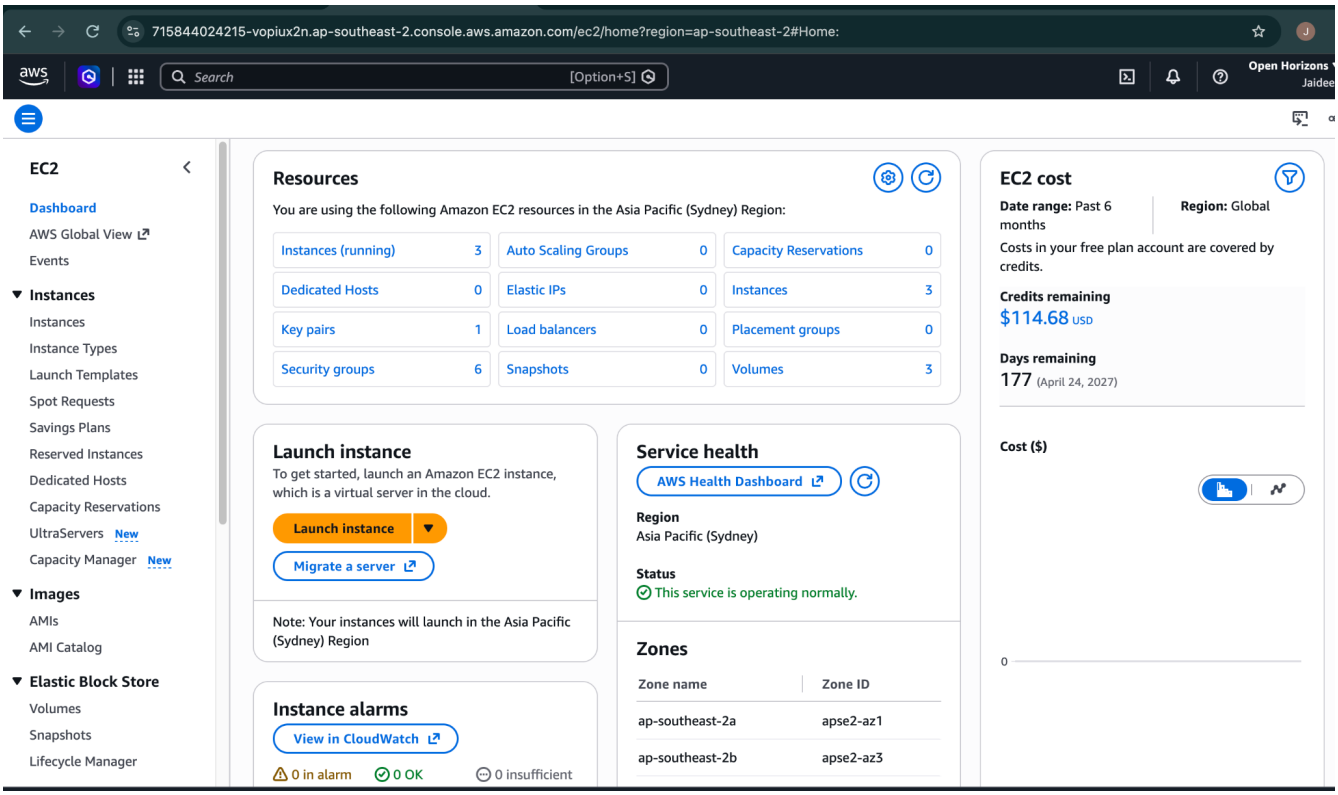


Host: AWS Console | **Test:** ewb-sg-app-b Inbound rules (2) after recovery — AWS console table / upper configuration | **Result:** SSH and restored ICMP rules displayed; restored ICMP rule ID sgr-0051e35a4a30d6de2



Host: AWS Console | **Test:** ewb-sg-app-b Inbound rules (2) after recovery — Selected resource details / lower panel | **Result:** SSH and restored ICMP rules displayed; restored ICMP rule ID sgr-0051e35a4a30d6de2

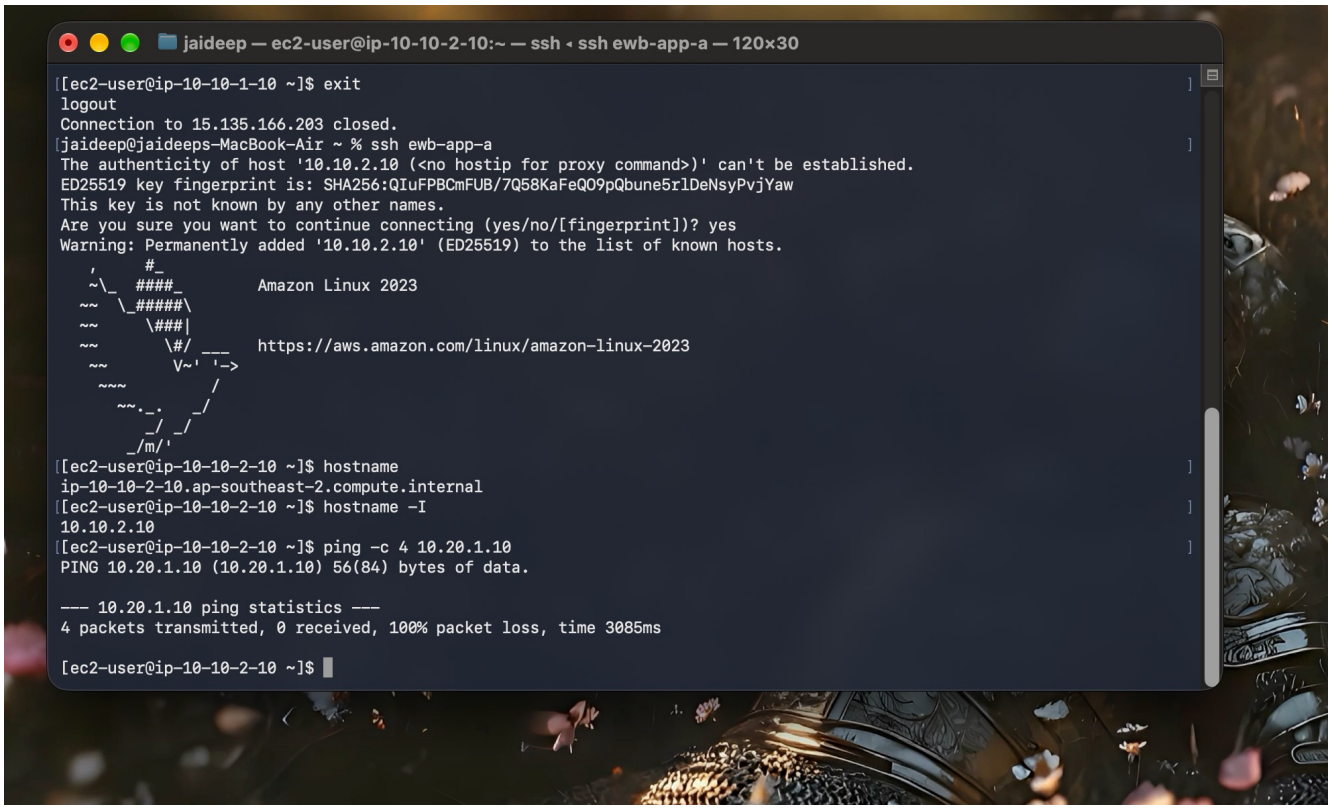
4e. EC2 dashboard before cleanup



Host: AWS Console | Test: EC2 dashboard before cleanup (3 instances running, 3 volumes, 1 key pair, 6 security groups)
Result: Dashboard screenshot shows the displayed resource counts.

5. Baseline failure

Before the route was restored, App-A could not ping App-B. This was a controlled baseline, not the final configuration.



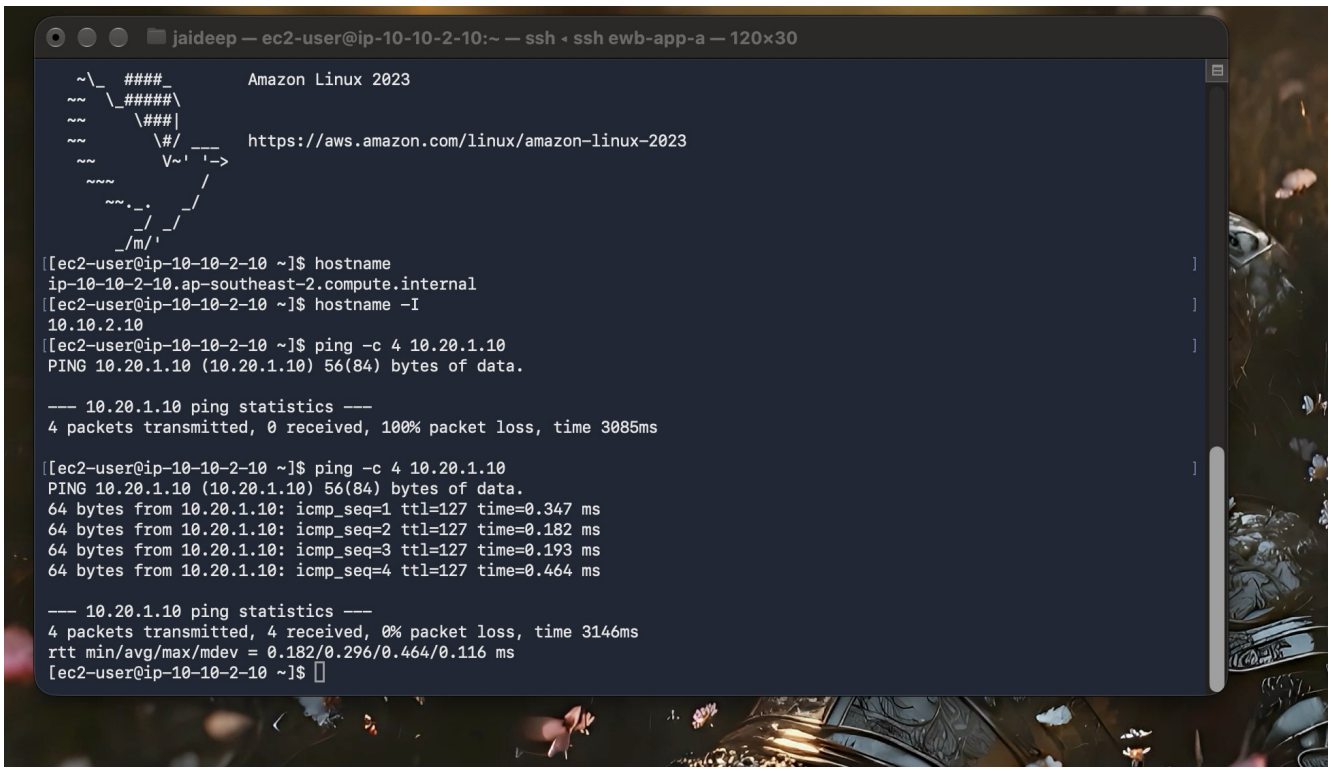
Host: App-A (10.10.2.10) | **Test:** ping -c 4 10.20.1.10 | **Result:** 4 sent, 0 received, 100% packet loss

> **Honest note:** I created this baseline by temporarily removing the pcx route from ewb-rt-a-private, then restored it. The AWS Console showed the green banner “Updated routes for ewb-rt-a-private.”

6. Forward ping and reverse ping

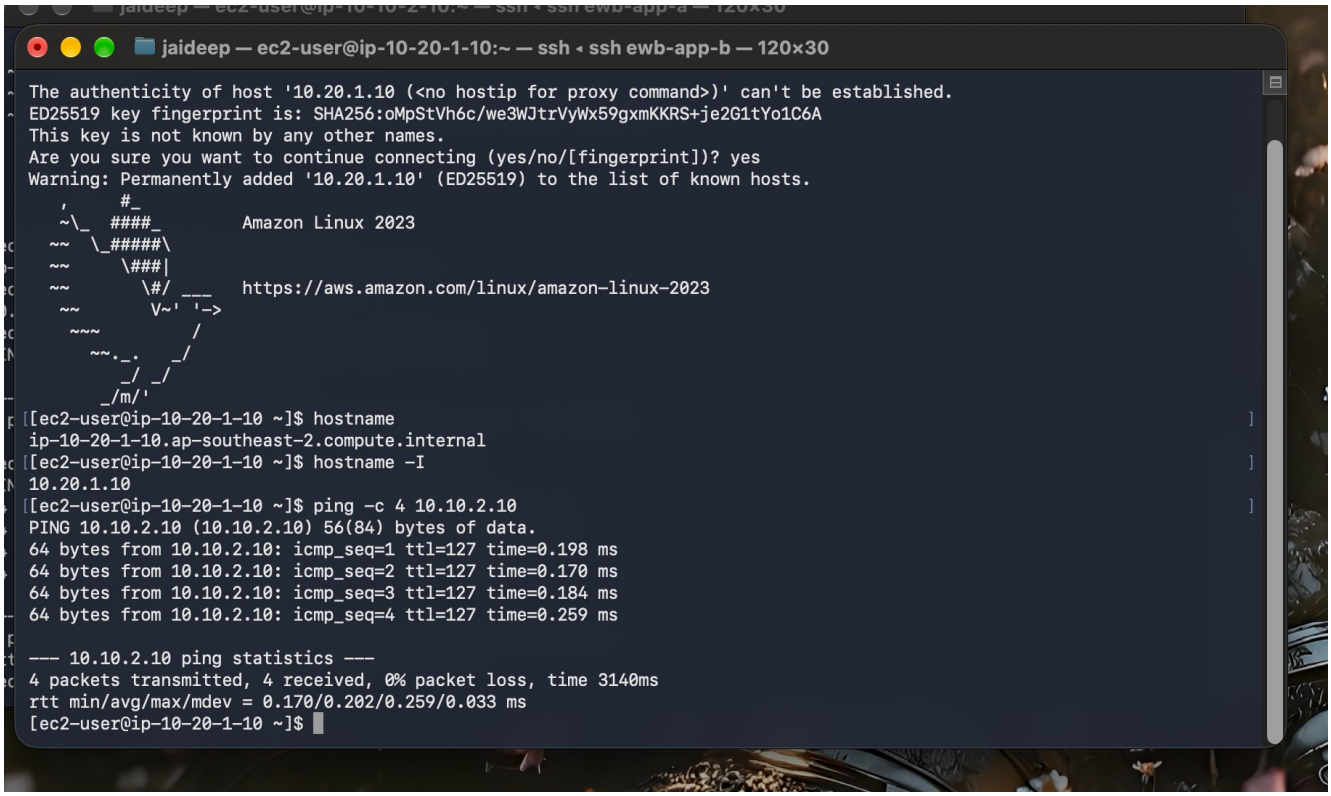
After the routes and required ICMP rules were restored, both directions were tested.

Forward — App-A to App-B



Host: App-A (10.10.2.10) | **Test:** ping -c 4 10.20.1.10 | **Result:** 4/4 received; 0% packet loss

6. Reverse ping — App-B to App-A

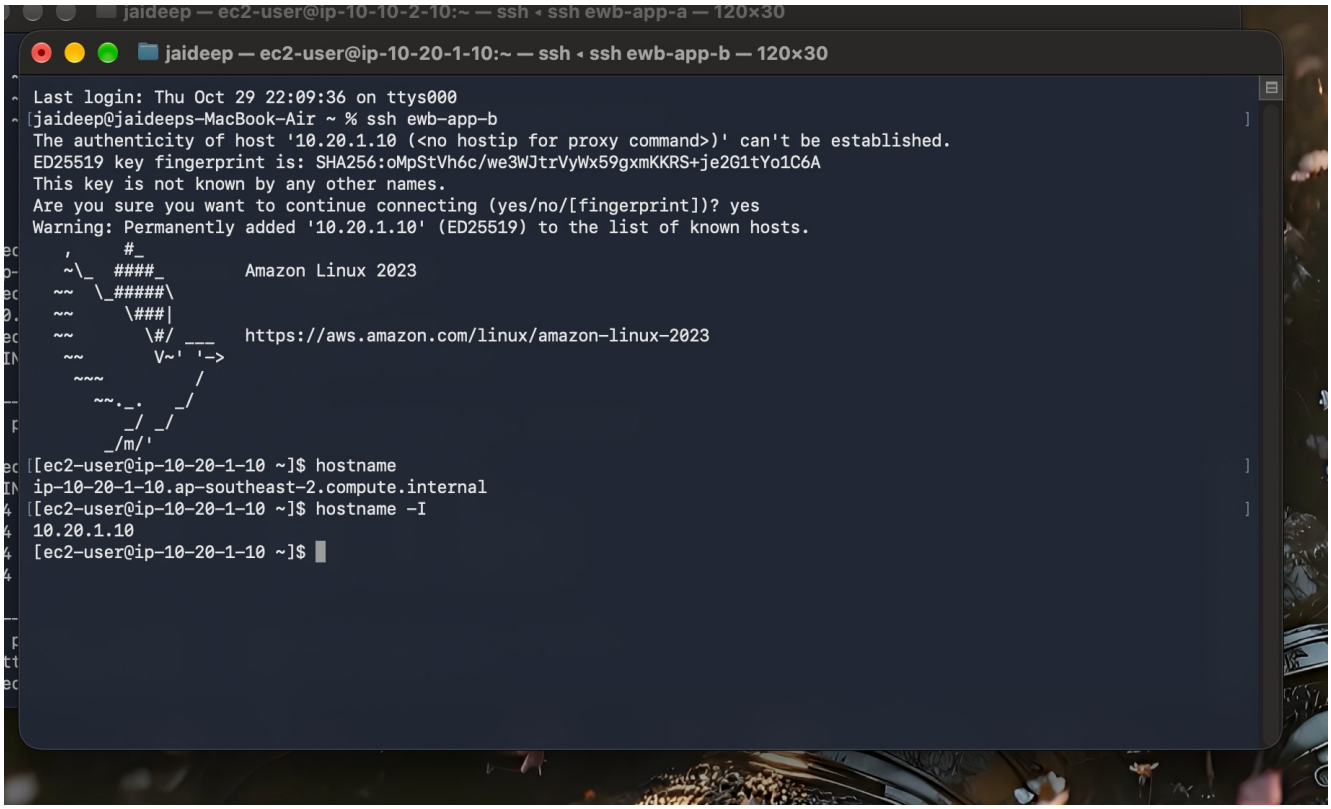


Host: App-B (10.20.1.10) | **Test:** ping -c 4 10.10.2.10 | **Result:** 4/4 received; 0% packet loss

These tests demonstrate private ICMP connectivity at the time of testing. They do not prove that every protocol is allowed.

7. App-B SSH proof

The SSH session reaches App-B through the bastion using ProxyJump. App-B reports hostname ip-10-20-1-10 and private IP 10.20.1.10.



Host: Bastion → App-B via ProxyJump | **Test:** SSH to ewb-app-b; hostname and IP check | **Result:** Hostname ip-10-20-1-10; private IP 10.20.1.10

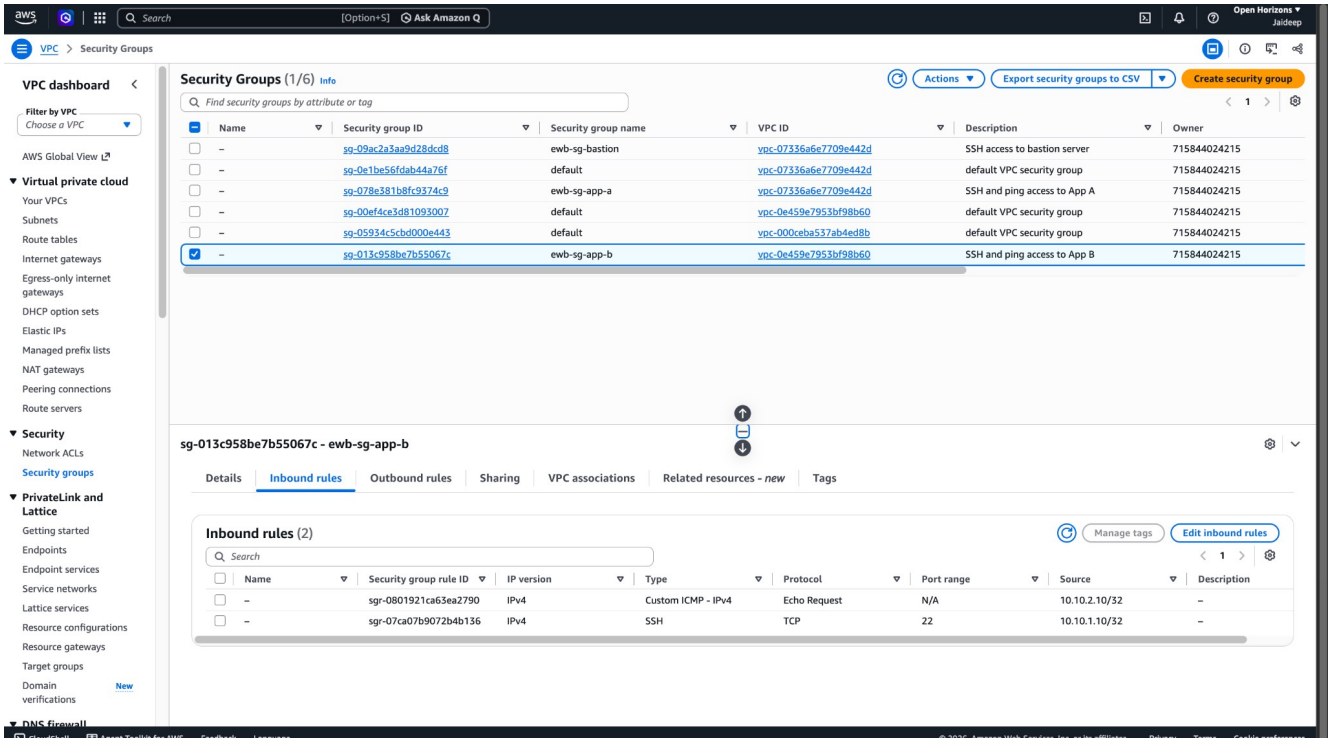
Why App-B allows SSH from 10.10.1.10

ProxyJump forwards the SSH connection through the bastion. App-B sees the bastion's private IP, 10.10.1.10, as the source—not the laptop's public IP.

8. Controlled failure and recovery

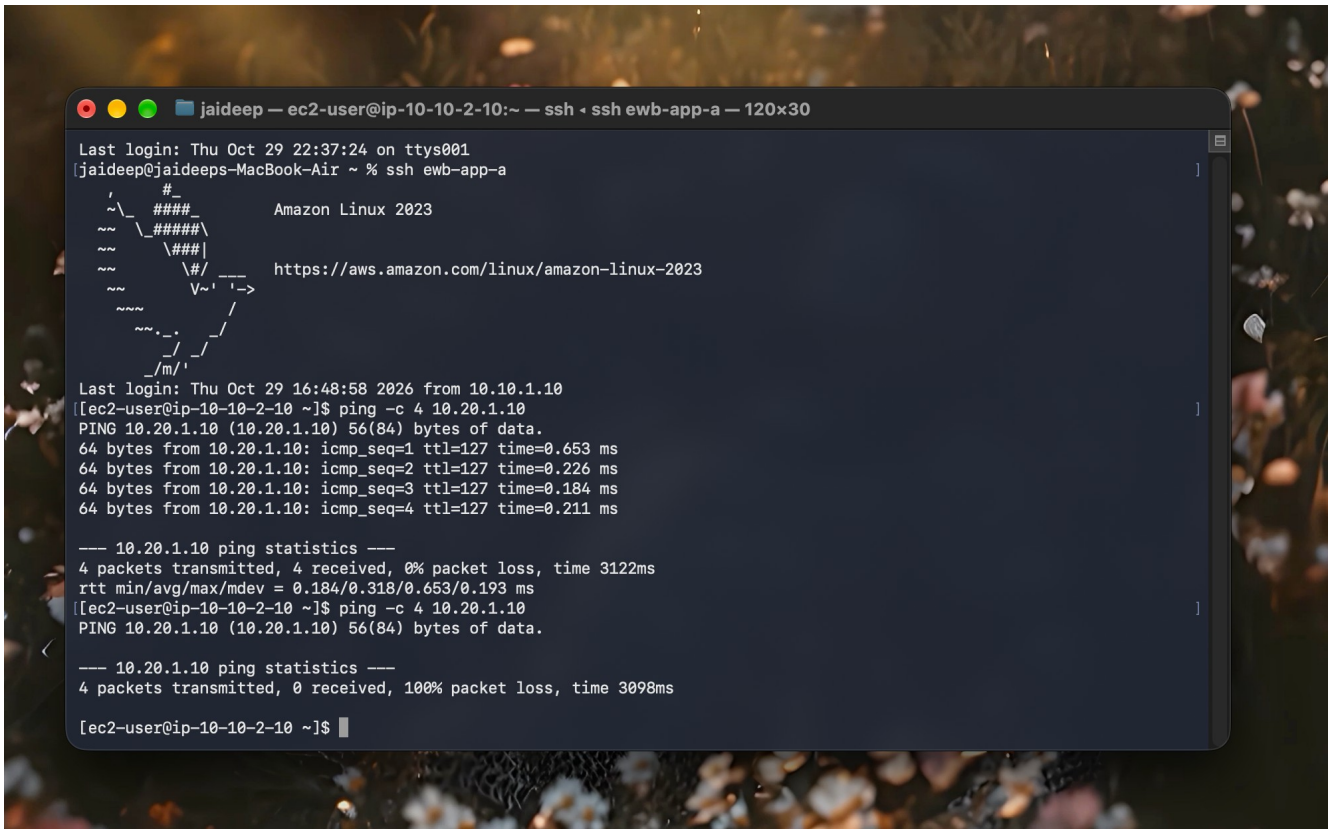
This test checks the App-B security group. The peering route remained in place; the ICMP Echo Request inbound rule was the rule being changed.

Before removal — two inbound rules



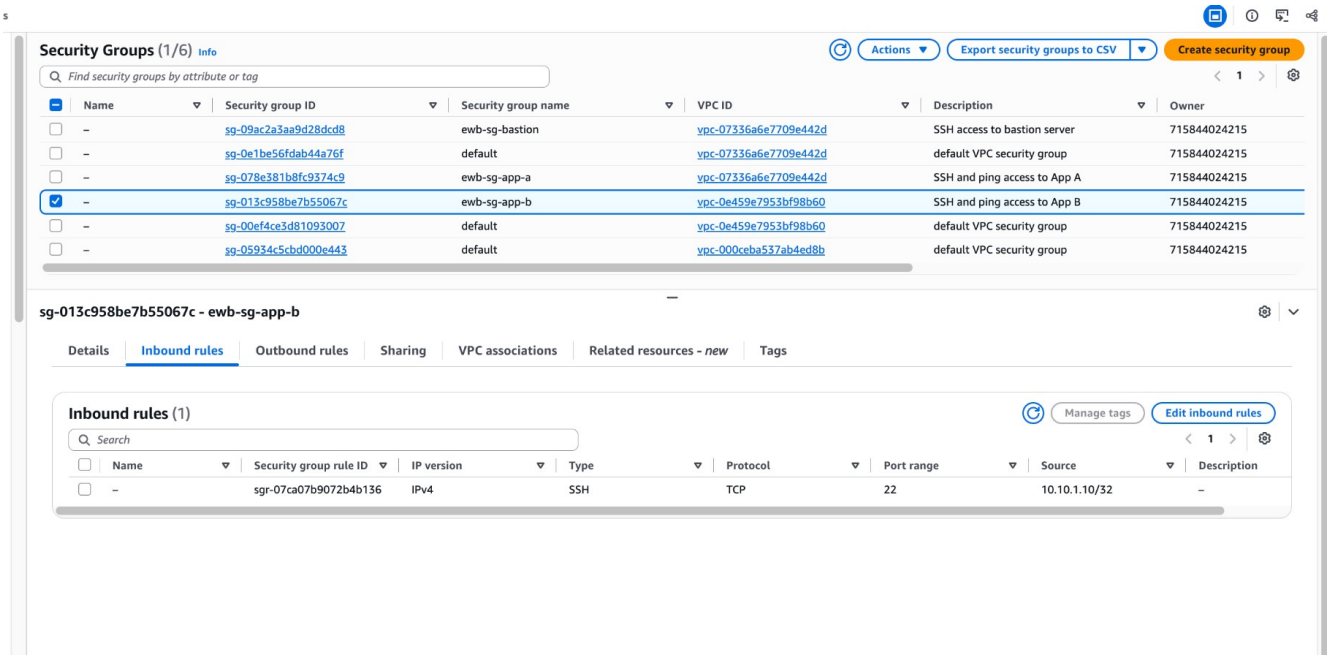
Host: AWS Console | **Test:** ewb-sg-app-b before ICMP removal | **Result:** Two inbound rules shown: SSH and ICMP

8. Controlled failure — ping after rule removal



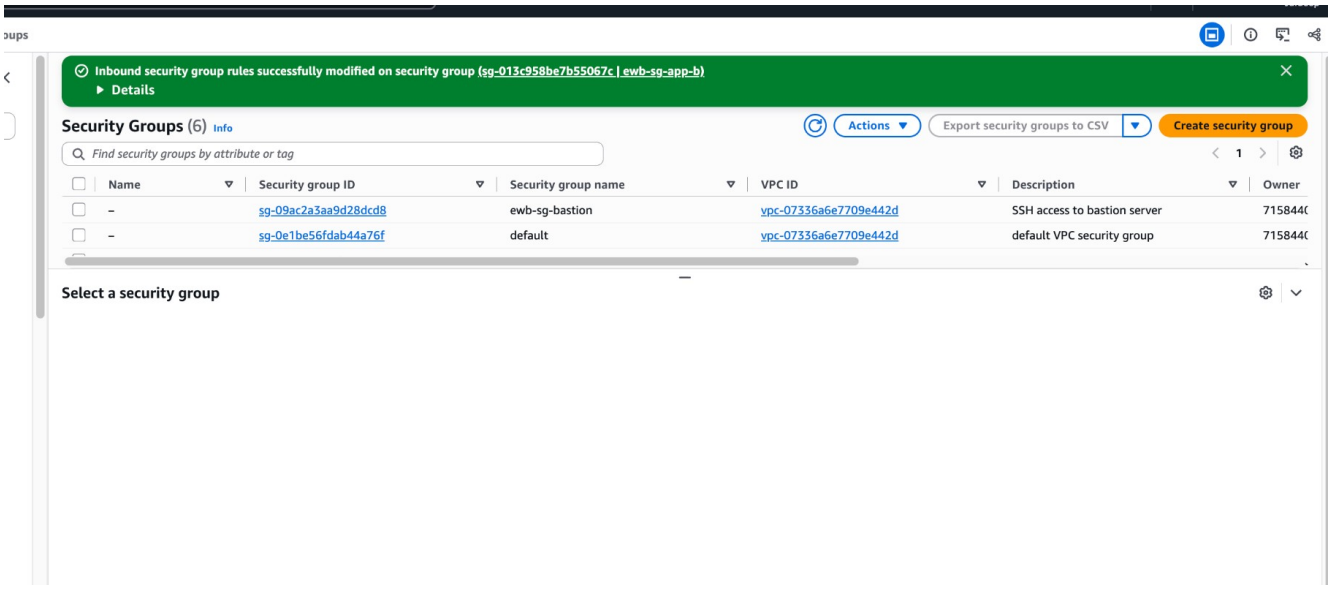
Host: App-A (10.10.2.10) | **Test:** Ping after removing App-B ICMP rule | **Result:** 4 sent, 0 received, 100% packet loss

8. After removal — App-B security group



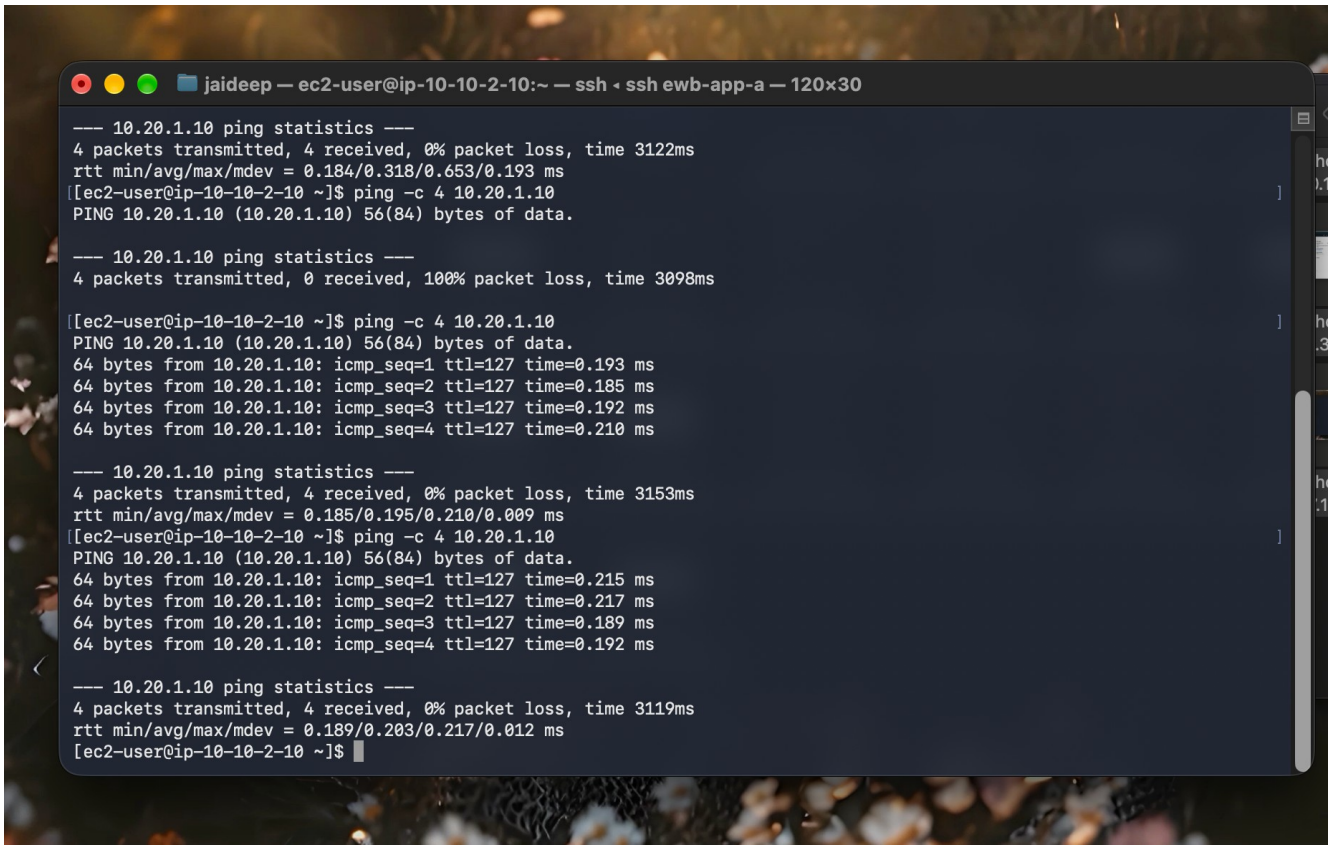
Host: AWS Console | **Test:** ewb-sg-app-b after ICMP removal | **Result:** One inbound rule shown: SSH only

8. Recovery — restore ICMP rule



Host: AWS Console | **Test:** Restore ICMP Echo Request on ewb-sg-app-b | **Result:** Green success banner confirms inbound security group rules were modified

8. Recovery — ping restored



Host: App-A (10.10.2.10) | **Test:** ping -c 4 10.20.1.10 after restoring ICMP | **Result:** 4/4 received; 0% packet loss

Conclusion

The route was fine during the controlled failure. Removing the ICMP inbound rule on App-B caused ping to fail. Restoring the rule allowed ping to succeed again.

9. Forward path and return path

Forward path: App-A (10.10.2.10) sends an ICMP Echo Request to App-B (10.20.1.10). The App-A private route table sends 10.20.0.0/16 traffic to pcx-0e12edd4a521208f3. App-B's security group must allow ICMP from 10.10.2.10/32.

Return path: App-B sends the Echo Reply to 10.10.2.10. VPC B's route table must send 10.10.0.0/16 traffic through the same peering connection. Both route tables need routes for two-way communication.

Why Active peering is not enough

Active means the peering connection is available; it does not automatically allow all traffic. Routes and security groups must permit the specific destination, protocol and source. Network ACLs and host firewalls can also affect traffic.

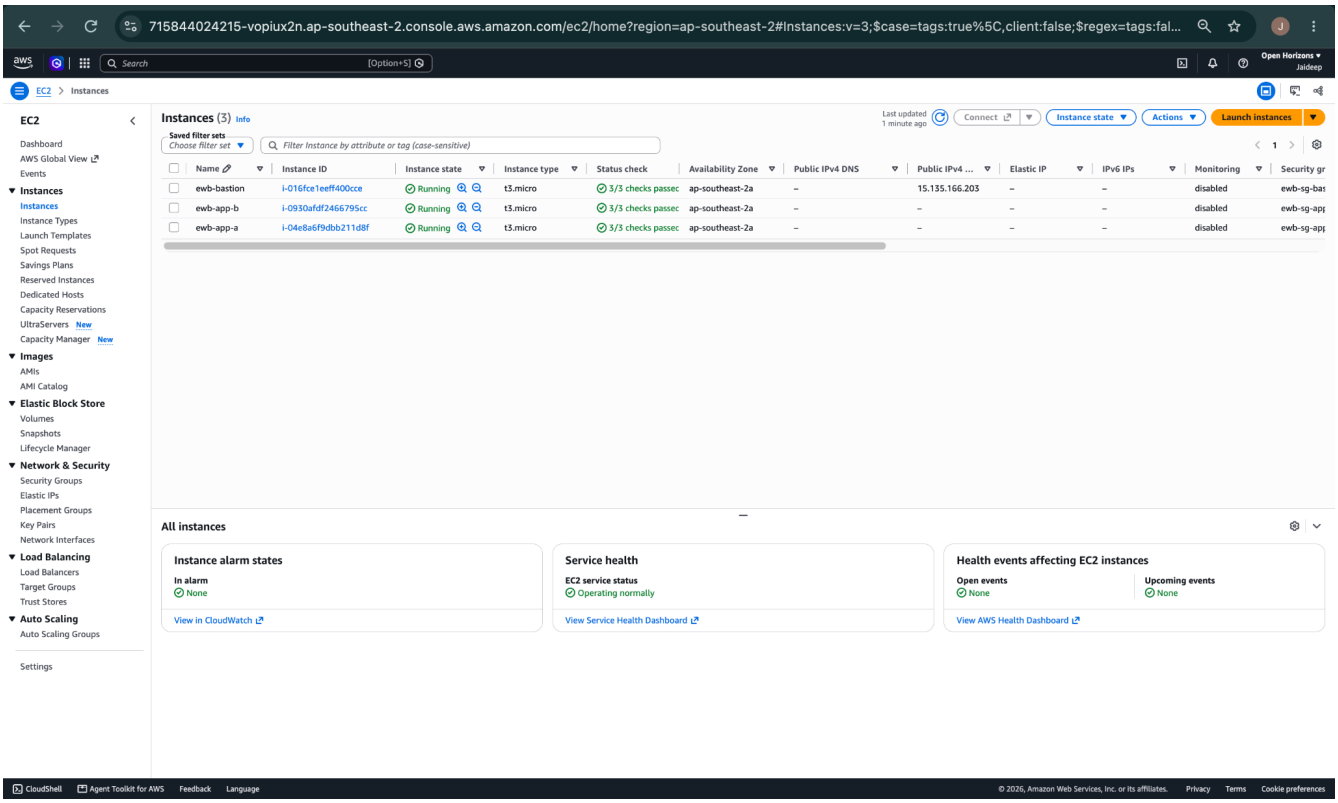
Why App-B SSH source is the bastion private IP

With ProxyJump, the bastion forwards the connection to App-B. App-B sees the connection from 10.10.1.10, so its SSH rule uses source 10.10.1.10/32.

Why VPC B cannot use VPC A's internet gateway over peering

VPC peering provides private IP connectivity; it does not make VPC A's internet gateway a shared internet exit for VPC B. VPC B needs its own egress design if outbound internet access is required. VPC B has no internet gateway.

9. Isolation proof



Host: AWS Console | Test: EC2 instances list; Public IPv4 column
Result: Bastion has a public IPv4; App-A and App-B show no public IPv4.

10. Cleanup

Cleanup status: Cleanup is planned within 2 days of submission. All resources carry the tag Project=EWB-VPC-Peering and will be removed in the order given on workbook page 14: terminate the 3 EC2 instances, confirm volumes are deleted, delete peer routes and ewb-a-b-peer, delete security groups, subnets and route tables, detach and delete ewb-igw-a, delete both VPCs, delete key pair ewb-lab and the local key, then verify with the project tag and check Billing. Cleanup evidence will be added after deletion.

11. What I learned

1. VPC peering connects private IP ranges across two VPCs.
2. Both VPCs need routes to the other VPC’s CIDR through peering.
3. Security groups control which sources and protocols can reach an instance.
4. Ping can fail because of a security group even when routes are correct.
5. ProxyJump lets me SSH to a private instance through a bastion host.
6. A successful ping proves ICMP connectivity at that time, not that all traffic is allowed.

Evidence note

The restored App-B ICMP rule is identified as sgr-0051e35a4a30d6de2. No private key, .pem content or password is included.